

RestaurantOS

Concept Prototype Walkthrough

Osteria Nove

A fictional Italian full-service restaurant, mid dinner service

Prepared for operator review

August 2026

Live demo: saengsawat.github.io/RestaurantOS

This is a working prototype built to start a conversation, not finished software. All data is fictional. Nothing is charged, printed, or sent anywhere.

How to use this document

RestaurantOS is a restaurant point-of-sale system in development. This guide walks through what the prototype does today, screen by screen, so you can judge whether it matches how a real dining room actually runs.

Open the live demo on a laptop, a tablet, or a phone. It works on all three. There is no login and nothing to install. The prototype opens mid service on purpose: tables are already seated, food is already in the kitchen, because an empty POS tells you nothing.

What we are asking you

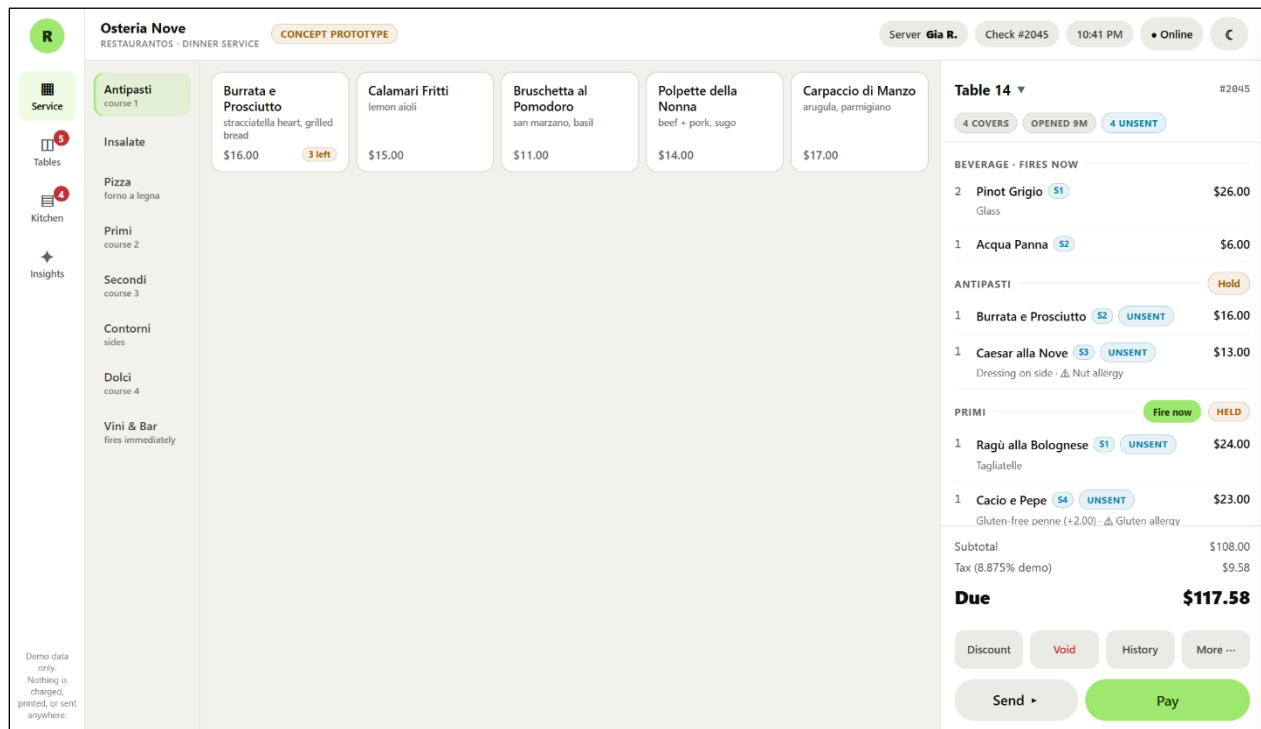
Not whether it looks nice. We want to know where it would break down on a Friday night, what a server or a line cook would find slow or wrong, and what is missing that gets touched every single shift. The last page lists the specific decisions we could not settle without an operator.

The four screens

- Service: take the order. Menu, the check, courses, seats, sending food to the kitchen.
- Tables: see the room. A floor plan showing who is seated, how long, and what stage they are at.
- Kitchen: cook the food. The kitchen display, one card per table, item by item.
- Insights: run the business. Where the same data turns into decisions. This is the part Toast and Square do not really do.

1. Service: taking the order

This is where a server spends most of the shift. Three panes: menu categories on the left, items in the middle, and the live check on the right.



The Service screen. Table 14 is mid order, with beverages already fired and the Primi course held back.

What it does

- Courses group themselves. Items land under Antipasti, Primi, Secondi, or Dolci automatically. Drinks fire to the bar immediately; food waits until you send it.
- Hold and fire. A held course shows a Fire now button, so the kitchen gets the pasta when the table is ready for it, not when it was rung in.
- Seat numbers. Every item carries a seat (S1, S2), which is what makes splitting a check by seat possible later.
- Allergies are loud. A nut or gluten flag rides with the item all the way to the kitchen ticket.
- Live stock. Burrata shows 3 left and counts down as it sells. At zero it 86s itself and greys out on the menu.
- Nothing disappears. A voided line stays visible with a Voided tag, struck through. The check tells the truth about what happened.

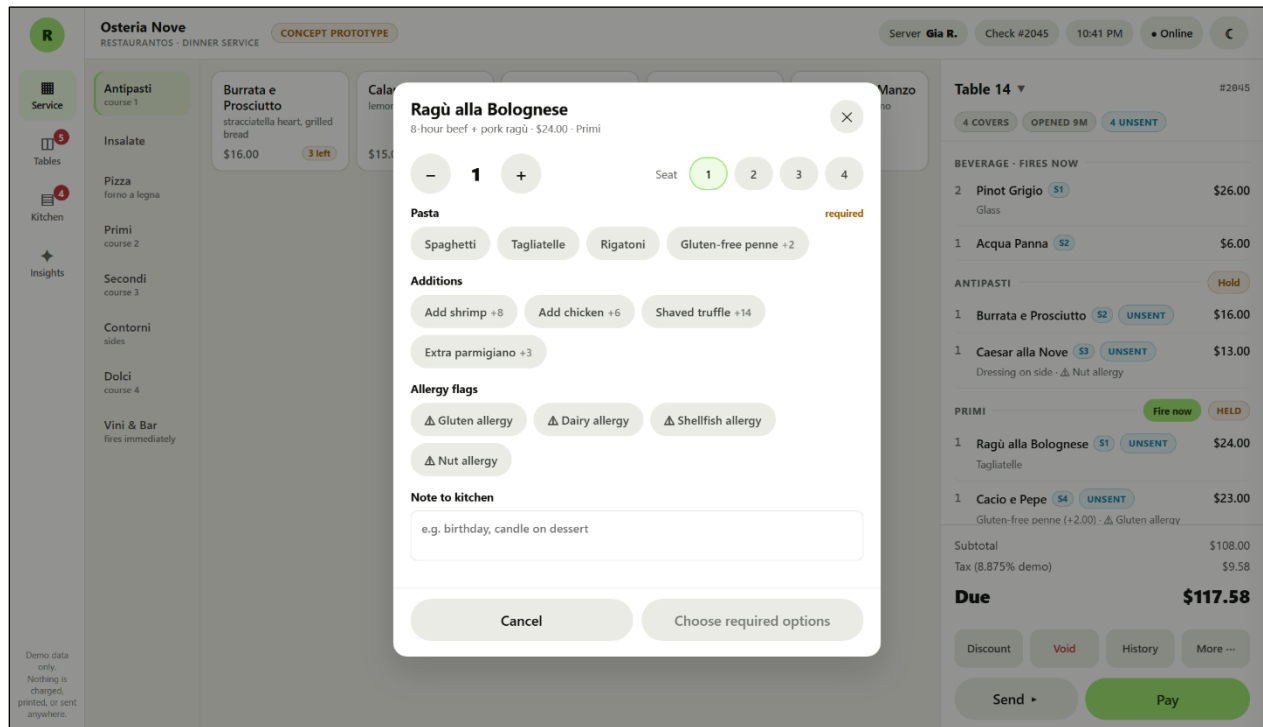
The buttons under the check

- Discount applies a comp or industry discount to the whole check.
- Void removes an item, but requires a reason and a manager PIN, and it is logged.

- History opens the full audit trail for the check: every item added, course fired, discount applied, and payment taken, with times.
- More holds table-level actions: guest count, transfer to another table, and a set of workflows we have deliberately not built yet.
- Send fires everything unsent. Pay opens payment.

Adding an item

Tapping a menu item opens its options. Required choices, such as the pasta shape, must be answered before the item can be added, so an incomplete order cannot reach the kitchen.



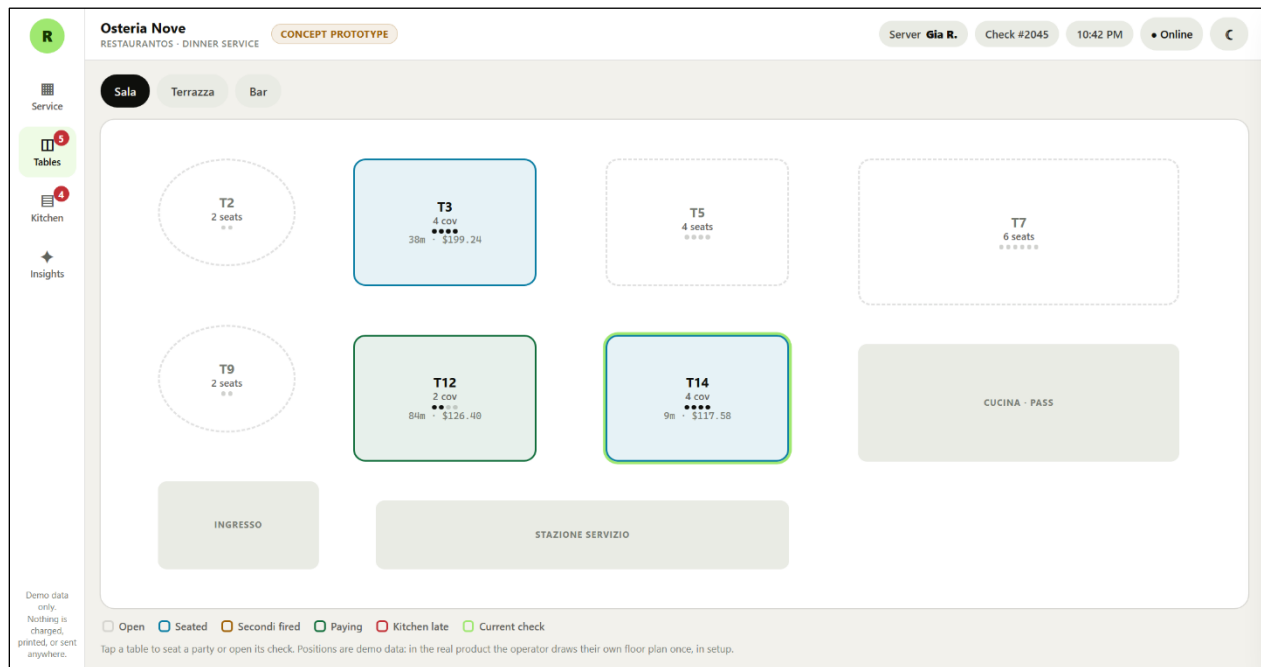
Ordering Ragù alla Bolognese: quantity, seat, required pasta choice, paid additions, allergy flags, and a free-text note to the kitchen.

Switching between tables

The table name at the top of the check is a button. Tap it to jump to any other open check, or to go seat a new table. If a server starts ringing on the wrong table, they are one tap from the right one, and the More menu can move the whole check to another table if items were already added.

2. Tables: seeing the room

The floor plan is laid out spatially, the way the dining room actually sits, rather than as a list of table numbers. Sections across the top switch between Sala, Terrazza, and Bar.



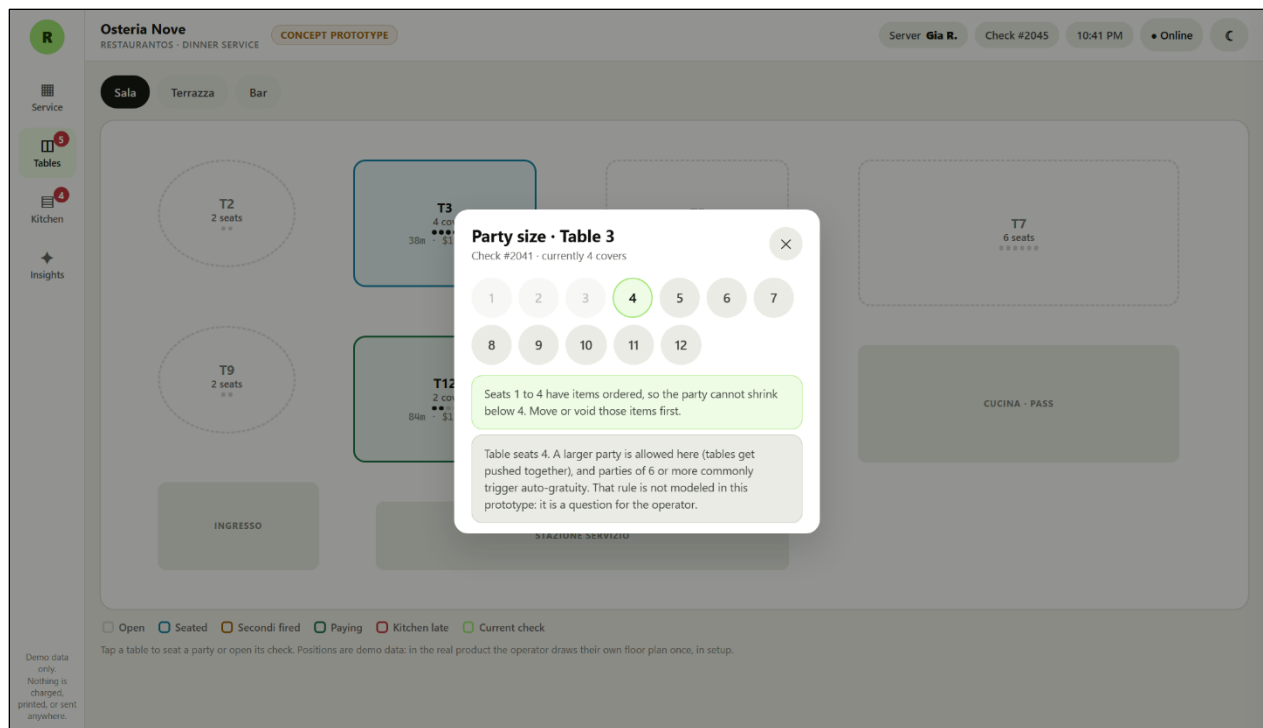
The Sala floor plan. Colour carries status, dots show seats filled against table capacity, and the kitchen pass and service station are drawn in for orientation.

What it does

- Status at a glance: dashed grey is open, blue is seated, amber means the mains have fired, green means they are paying, red means the kitchen is running late on that table.
- Each occupied table shows covers, how long they have been sitting, and the running total.
- Seat dots show party size against table capacity, so a two-top holding four is obvious.
- Tap a table to seat a party, open its check, change the guest count, or transfer it.
- In the real product the operator draws their own room once during setup. The layout here is data, not code.

Changing the party size

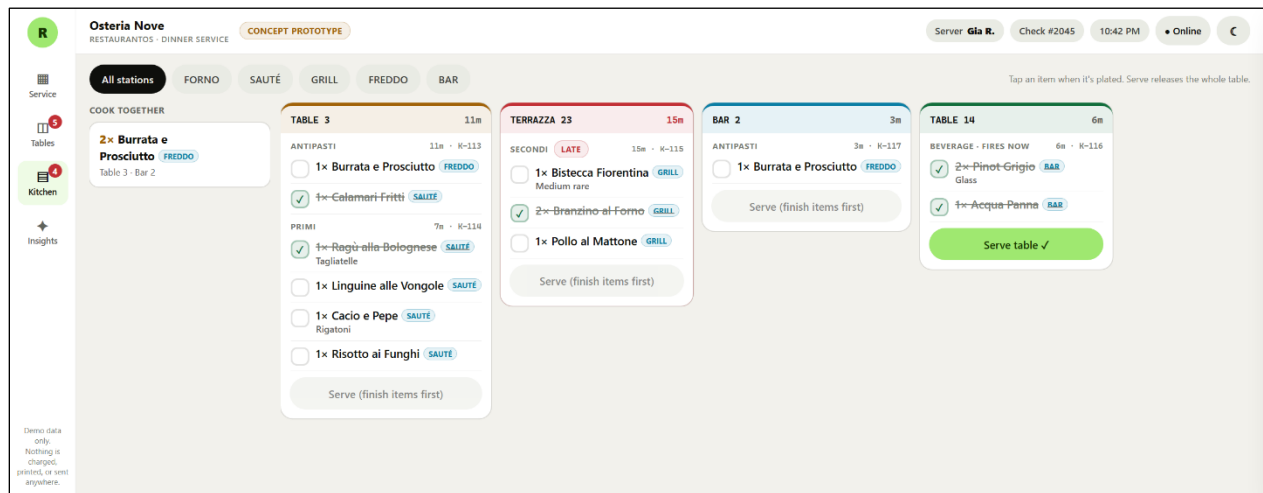
Two more guests join a four-top. Guest count is editable after seating, from the table or from the More menu on the check.



Changing party size. Covers cannot drop below a seat that already has items ordered, and the prototype flags the auto-gratuity question rather than guessing at your policy.

3. Kitchen: the line

The kitchen display was rebuilt after feedback from three years of sous chef experience. It is organised the way a kitchen actually thinks, by table, not by ticket.



The kitchen display, all stations. Table 3 has two courses stacked on one card, Terrazza 23 is late, Table 14 is fully plated and ready to serve.

What it does

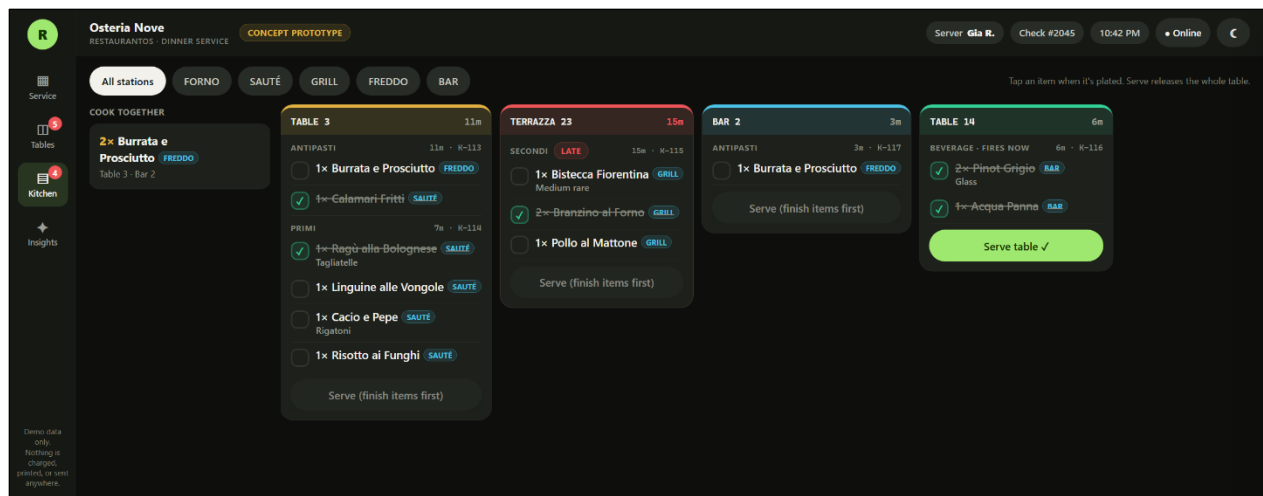
- One card per table. When a table fires a second course, it stacks onto the same card instead of adding a new ticket to the rail. Anything just fired is tagged New.
- Items are ticked off one at a time. Tap an item when it is plated. Tap again if you tapped it by mistake.
- The card colour follows the work: blue is untouched, amber is partly plated, green is fully plated and ready, red is past the late threshold.
- Serve releases the whole table, and only unlocks once every item is done. It appears on the all stations view only, because releasing a table is expo's call, not one station's.
- A served table stays recallable for ten minutes. A mis-tap during a rush is recoverable.
- Cook together, on the left. Identical dishes across different tables are grouped, so the line fires two burrata at once instead of hunting through cards. That removes a job someone is doing by hand today.

Station filters

The tabs across the top filter to a single station. A cook on saute sees only saute items. Station views cannot serve a table, they are for cooking only.

Night mode

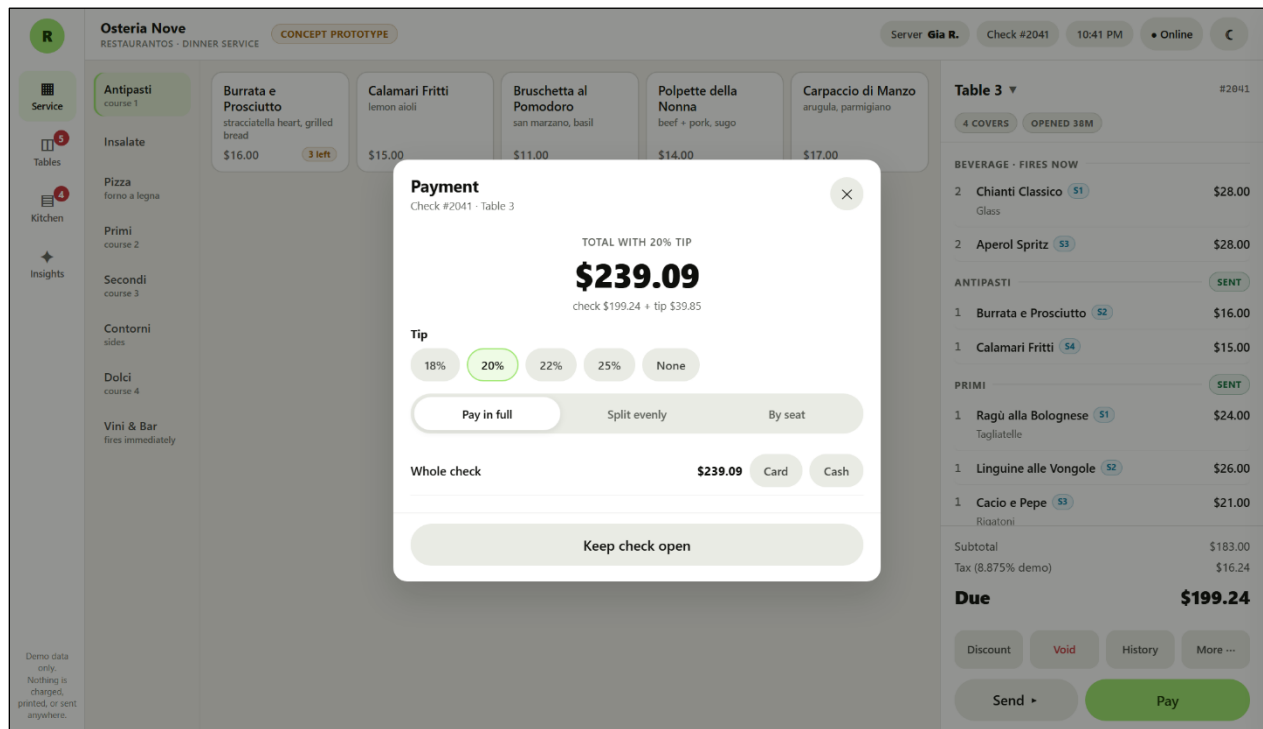
Half of service happens in a dim room. The whole system has a night theme, and the kitchen is where it matters most.



The same kitchen display in night mode. Warm blacks, not blue-blacks, with the status colours brightened to hold up on a dark screen.

4. Payment and splitting

Payment opens from the check. The prototype will not let a check be paid while items are still unsent, because food that never reached the kitchen should not be billed.



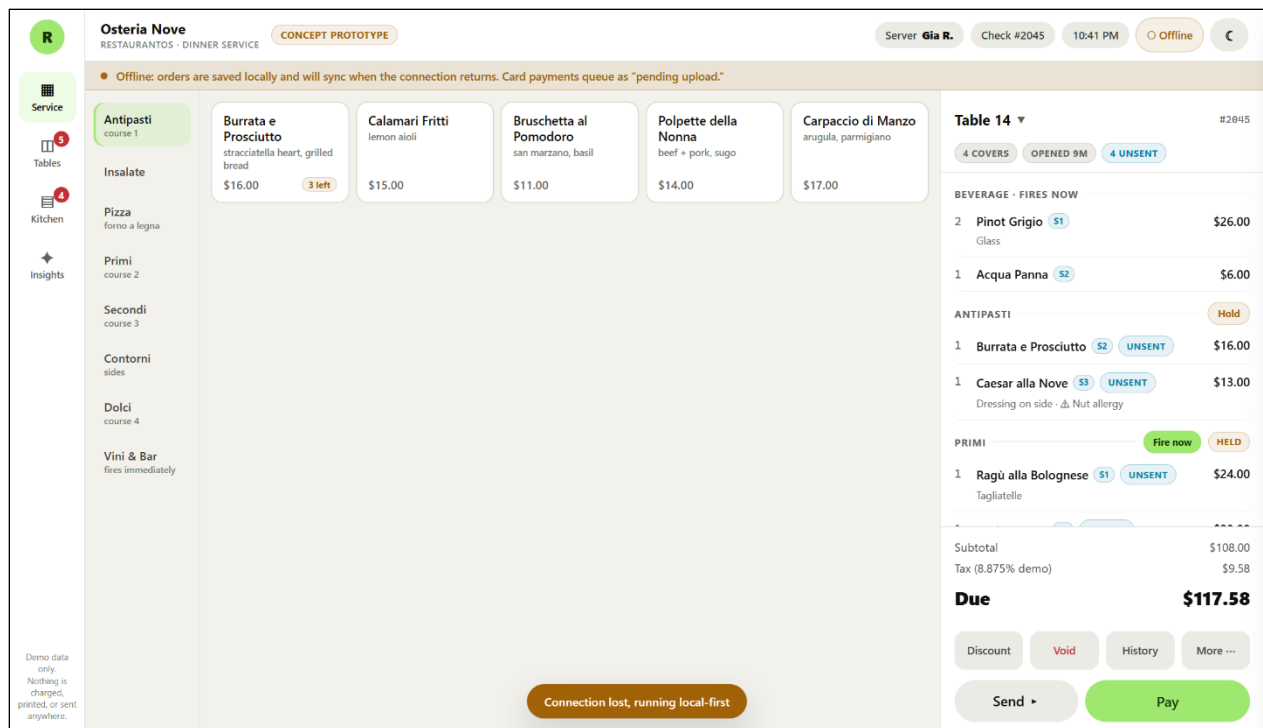
Paying Table 3. Tip percentages, three split modes, and per-guest card or cash buttons.

What it does

- Three ways to split: pay in full, split evenly between two and six guests, or split by seat using the seat numbers from the order.
- Split by seat charges each guest for what they actually ate, not an average.
- Cash and card can be mixed on one check.
- The check will not close until the full amount is settled.
- Splitting to the cent is handled properly. Money is never allowed to appear or vanish in a split.

When the internet goes down

Tap the Online button in the top bar to simulate an outage. This is the part most worth your judgement, because it is where a POS either keeps a restaurant running or stops it.

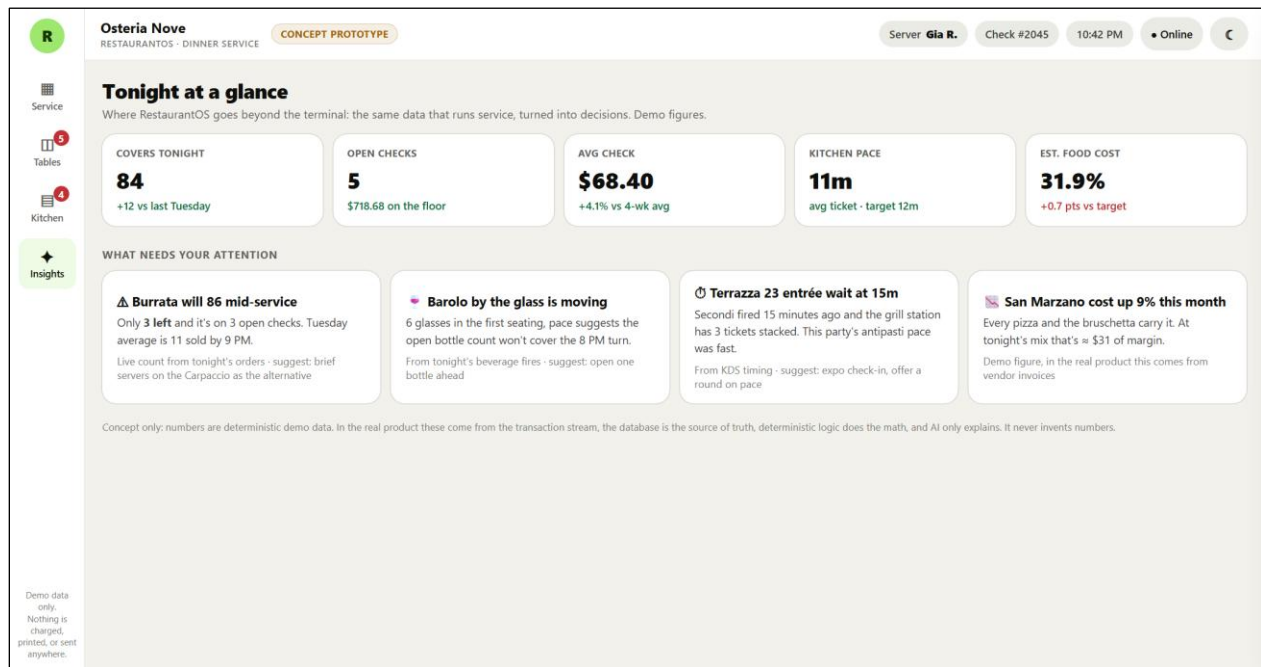


Offline. Orders keep working and save locally, and the banner says plainly what is happening.

- Orders, sending to the kitchen, and cash all keep working while offline.
- A card taken offline shows Pending upload, never Paid. It has not been authorised yet, and the prototype refuses to pretend otherwise.
- When the connection returns, queued work syncs and the system says so.
- This honesty is deliberate. A POS that shows a payment as complete when the processor never saw it is how restaurants lose money.

5. Insights: the part that is different

Toast and Square both give you reports. Reports tell you what happened last month. This screen is an early sketch of something else: the same data, read live, turned into decisions you can act on tonight.



The Insights screen, reading live from the same service data shown on the other screens.

What it is showing

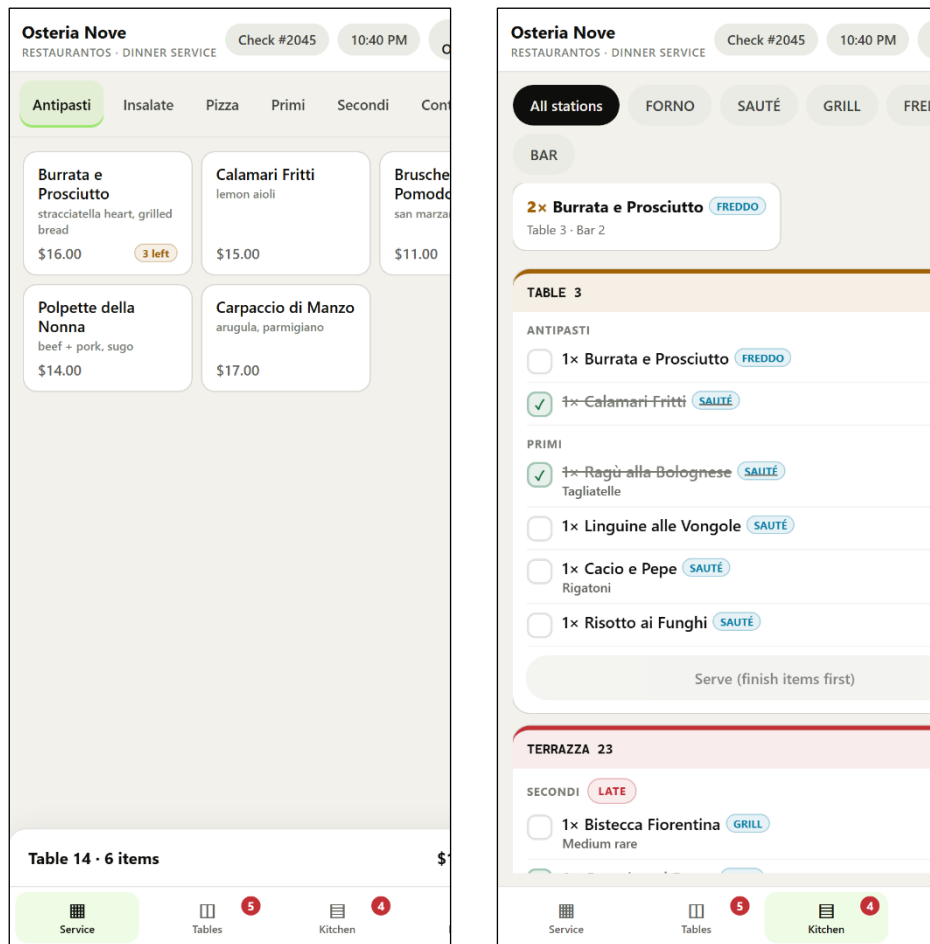
- Tonight's numbers: covers, open checks and what is on the floor, average check, kitchen pace against target, estimated food cost.
- What needs your attention: specific, actionable observations rather than charts.
- The burrata warning counts the actual stock left and the actual open checks it is on. It is live, not a canned example.
- Each card explains where it came from, so a recommendation can be trusted or argued with.

Where this goes

This is the smallest possible taste of the direction. The longer plan is food cost, labour, and purchasing intelligence built on the restaurant's own transaction data: what happened, why, what is likely next, and what to do about it. The principle we hold to is that the database is the source of truth and the maths is deterministic. The AI explains, it never invents a number.

6. On a phone

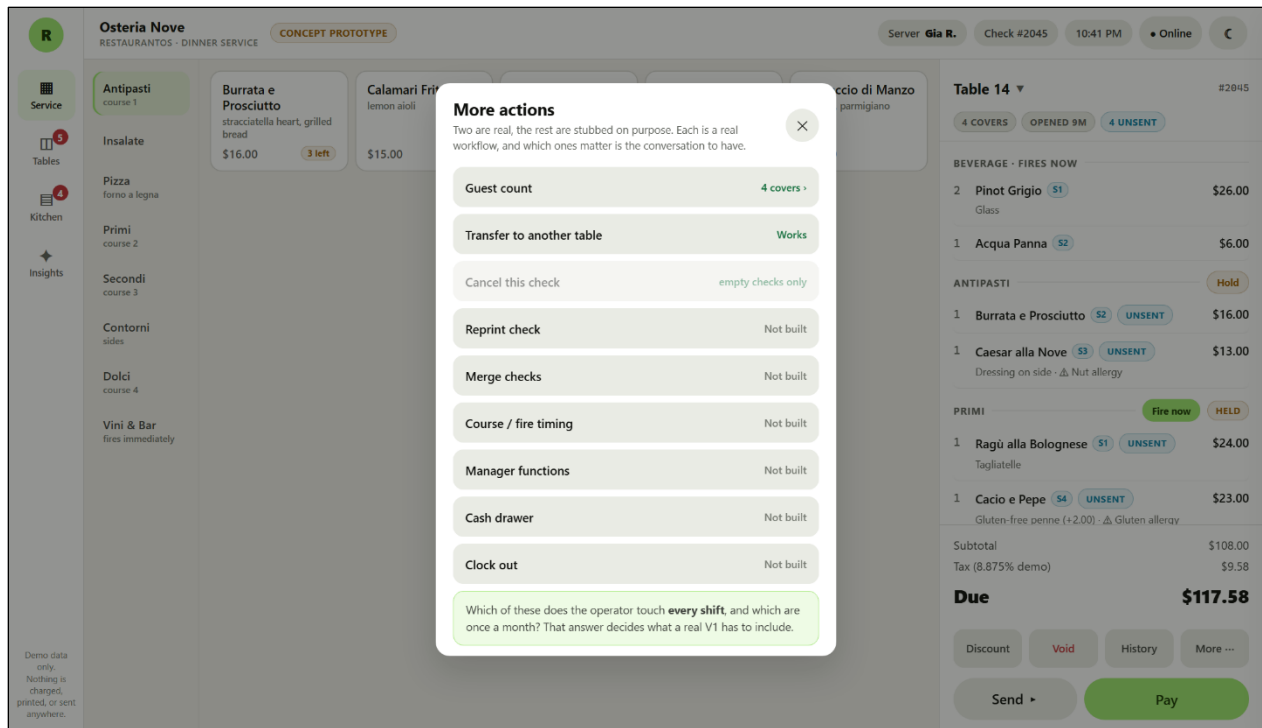
The same system reshapes itself for a handheld. Navigation moves to the bottom, and the check becomes a sheet that slides up from a bar showing the table and the amount due.



Service and Kitchen on a phone. Bottom tabs, full width cards, and the check as a slide-up sheet.

What we have deliberately not built

The More menu lists real workflows that are stubbed on purpose: reprinting a check, merging checks, fire timing, manager functions, the cash drawer, and clocking out. Which of these you touch every shift, and which you touch once a month, decides what has to be in the first real version.



The More menu. Two actions work; the rest are named honestly as not built.

7. What we need from you

Everything below is a judgement call we made without an operator in the room. Each one is cheap to change now and expensive to change once it is built. Blunt answers are the most useful kind.

The big one

When the internet goes down, must orders still reach the kitchen? If yes, the system needs extra hardware in the restaurant and that decision has to be made before anything is built. It is close to impossible to add later.

Service and the floor

- Tapping an occupied table currently shows a small menu first. Would you rather it opened the check straight away?
- Are courses and hold or fire used routinely, or does everything just go when it is rung?
- Is seat level ordering worth the extra taps, or do servers work by position at the table?
- What split behaviour is genuinely mandatory on night one?
- Should guest count be required before a table can be seated?
- Is auto-gratuity used, and from what party size?

The kitchen

- Is one card per table right, or do cooks want a ticket per fire?
- How long should a newly fired course stand out as New? We guessed three minutes.
- Should a station be able to bump a table out, or only expo?
- How long should a served table stay recallable? We guessed ten minutes.
- If a table is fully plated but nobody has run the food, should the screen start shouting? Right now it stays quiet.
- Does the cook together panel help, or is it noise?

Money and the close

- Are tips adjusted after the card is authorised?
- Are bar tabs and card preauthorisation required?
- What is the largest card amount you would accept offline, knowing it might decline later?
- What does closing the day actually mean in your restaurant?
- Which actions should need a manager?
- How are voids handled today when the food has already been cooked?

And the honest question

Looking at all of it: what is missing that you touch every single shift, and what here would you never use? We would rather cut half of this than build the wrong half.

Thank you. Your answers set the direction for the real build.

RestaurantOS concept prototype, August 2026. All restaurant data in this document is fictional.